Guía de Google Cloud + Workspace para IM3

Cómo funciona lo que montamos y cómo replicarlo a clientes

> ***Audiencia**: Mateo y el equipo de IM3. **Propósito**: entender en profundidad la infraestructura que conecta nuestro código con Google (Gmail, Drive, Calendar) para poder replicarla a clientes con confianza. **Aclaración**: NO es manual oficial de Google. Es la versión práctica, aterrizada a lo que usamos en este proyecto.*

Índice

1. El Modelo Mental de 4 Capas
2. Google Workspace vs Google Cloud vs Gmail Personal
3. Organización e IAM Hierarchy
4. Service Accounts a fondo
5. Domain-Wide Delegation — La Magia
6. Las APIs de Google Workspace
7. gcloud CLI — Tu Herramienta Principal
8. Anatomía de Este Proyecto (IM3)
9. Playbook de Replicación a Cliente Nuevo
10. Seguridad y Recovery
11. Troubleshooting Frecuente
12. Glosario

---

1. El Modelo Mental de 4 Capas

La mayor confusión con Google viene de mezclar cosas que son independientes. Hay **4 capas**, y cada una tiene su propio sistema de permisos:

```
┌──────────────────────────────────────────────────────────┐
│  Capa 1: GOOGLE ACCOUNT (identidad de una persona)          │
│  Ejemplos: mateoobandoangel@gmail.com, info@im3systems.com  │
│  Una persona real o un buzón. Se loguea con contraseña+2FA. │
└──────────────────────────────────────────────────────────┘
                           │
                           │ se agrupa en
                           ▼
┌──────────────────────────────────────────────────────────┐
│  Capa 2: GOOGLE WORKSPACE (antes G Suite)                   │
│  Dominio: im3systems.com                                    │
│  Producto de pago que gestiona los emails @im3systems.com   │
│  Define: usuarios, grupos, políticas, Admin Console         │
│  Quien paga: el dueño de la empresa                         │
└──────────────────────────────────────────────────────────┘
                           │
                           │ cada workspace tiene asociada UNA
                           ▼
┌──────────────────────────────────────────────────────────┐
│  Capa 3: GOOGLE CLOUD ORGANIZATION                          │
│  Nombre: "im3systems.com" (ID: 980700249203)                │
```

```
│  Contiene folders, proyectos, recursos de Cloud              │
│  Permite políticas globales (ej: quién puede crear proyectos)│
└──────────────────────────────────────────────────────────────┘
                           │
                           │ dentro de la org viven N
                           ▼
┌──────────────────────────────────────────────────────────────┐
│  Capa 4: GOOGLE CLOUD PROJECT                                 │
│  Nombre: "im3-diagnostics" (ID: 634399080625)               │
│  Aquí viven: APIs habilitadas, Service Accounts, Billing,   │
│  Logs, quotas, BigQuery datasets, etc.                        │
│  Dentro del proyecto: SERVICE ACCOUNTS (identidades robot)   │
└──────────────────────────────────────────────────────────────┘
                           │
                           ▼
        Dentro del proyecto existen SERVICE ACCOUNTS:
        im3-diagnostics@im3-diagnostics.iam.gserviceaccount.com
        (robot que ejecuta tu código)
```

Cada capa tiene SU propio IAM

No es el mismo IAM. Cuando miraste las 2 screenshots en nuestra sesión anterior, una era IAM a nivel **organización** y otra a nivel **proyecto**. Son listas distintas.

Capa	Dónde se gestiona	Qué controla
Google Account	https://myaccount.google.com	Contraseña, 2FA, recovery
Workspace	https://admin.google.com	Usuarios del dominio, Apps instaladas, Domain-Wide Delegation
Org (Cloud)	Cloud Console → IAM → selector "Organization"	Policies globales, quién puede crear proyectos
Project (Cloud)	Cloud Console → IAM → selector "Project"	Quién tiene qué rol en ese proyecto

Regla de oro

Los roles se HEREDAN hacia abajo, nunca hacia arriba. Si te doy "Owner" en la organización, eres Owner de TODOS los proyectos debajo. Si te doy "Owner" en un proyecto, no tienes nada en otros proyectos ni en la org.

---

2. Google Workspace vs Google Cloud vs Gmail Personal

Gmail Personal (`@gmail.com`)

- Gratis.
- Cuenta de una sola persona.
- NO puede ser super admin de un dominio.
- NO puede usar Domain-Wide Delegation.

- Útil para uso personal, no para negocio serio.

Google Workspace (`@tudominio.com`)

- **Producto de pago** (~$6-18/usuario/mes).
- Te da: Gmail empresarial, Calendar, Drive, Meet, admin console, DWD, políticas.
- **Es donde `info@im3systems.com` vive**.
- Lo administras desde admin.google.com.
- Facturación separada de Google Cloud.

Google Cloud

- Servicio de infraestructura en la nube (IaaS/PaaS).
- Te da: APIs, compute, storage, databases, BigQuery, Vertex AI, etc.
- Proyectos, service accounts, billing granular por proyecto.
- Lo administras desde console.cloud.google.com.
- **Puede existir SIN Workspace** (cuenta personal puede crear proyectos Cloud).
- **Pero se hace poderoso cuando se combina con Workspace** via Domain-Wide Delegation.

La relación

```
Workspace (admin.google.com)          Google Cloud (console.cloud.google.com)
────────────────────────────          ───────────────────────────────────────
Usuarios del dominio                  Proyectos + APIs
Email, Calendar, Drive                Service Accounts, Billing
Domain-Wide Delegation setup  ◄──────  Se "registra" aquí (por Client ID)
```

Cuándo crear qué

- Cliente quiere integrarse con Gmail/Drive/Calendar propio → **NECESITA Workspace** (no gmail personal) + tú creas un proyecto Cloud tuyo o suyo
- Cliente quiere usar IA/BigQuery/Cloud SQL → **NECESITA Google Cloud** (con o sin Workspace)
- Cliente solo necesita que le mandes emails → NO necesita Workspace, tú usas Resend/SendGrid

---

3. Organización e IAM Hierarchy

Cuando un cliente compra Workspace, Google auto-crea una "organización" en Cloud bajo ese dominio — pero queda **inactiva** hasta que se activa Cloud.

Jerarquía

```
Organization (im3systems.com)
├── Folder (opcional, para agrupar por equipo/cliente)
│   ├── Project A
│   │   ├── Service Account 1
│   │   ├── API habilitada: Calendar
│   │   └── API habilitada: Drive
│   └── Project B
└── Project C (sin folder)
```

Roles — qué significa cada uno

Los 3 roles "Basic" (los más comunes):

- **Owner**: puede hacer TODO + gestionar billing + añadir otros admins
- **Editor**: puede modificar recursos, NO puede cambiar billing ni IAM
- **Viewer**: solo lectura

Roles específicos útiles:

- **Project Creator** (a nivel org): puede crear proyectos dentro de la org
- **Project Mover** (a nivel org + project): puede mover proyectos entre orgs
- **Service Account User**: puede "usar" service accounts (ej: para deploy)
- **Service Account Token Creator**: puede impersonar service accounts

Domain Restricted Sharing — La política que te bloqueó

Cuando intentaste agregar tu personal a la org, Google respondió:

> *`IAM policy update failed: Domain Restricted Sharing enforced`*

Esta es una **organization policy** (`constraints/iam.allowedPolicyMemberDomains`) que **restringe qué dominios pueden recibir permisos** dentro de la organización. Por default en orgs nuevas, solo permite el propio dominio (`im3systems.com`).

Esto es bueno — previene que por error añadas un `@gmail.com` random con acceso a tu infra.

Cómo revisarla: https://console.cloud.google.com/iam-admin/orgpolicies (con la org seleccionada arriba)

Cómo modificarla (si SÍ necesitas agregar externos, ej: consultor temporal):

1. Editar la policy
2. Añadir dominios permitidos específicos o "Allow all" (no recomendado)
3. Save

Lección para ti

Si un cliente te contrata como consultor, NO te añadas como principal en su org (choca con esta policy). En su lugar:

- Opción 1: el cliente te crea un email en su workspace (ej: `mateo@cliente.com`)
- Opción 2: trabajas vía el service account (no necesitas tu email ahí)

---

4. Service Accounts a fondo

Un **service account** es una identidad que usa tu código (no un humano) para llamar APIs de Google. Piensa en él como "el robot que corre en producción".

Por qué usar service accounts

- Tu código no se loguea con contraseña humana (peligroso y frágil)
- El service account tiene permisos específicos y auditables
- Si lo comprometen, rotas su key sin afectar humanos
- No se cae si tú te vas de vacaciones

Anatomía

Cuando lo creas, te da:

- **Email**: `NOMBRE@PROJECT_ID.iam.gserviceaccount.com` (ej: `im3-diagnostics@im3-diagnostics.iam.gserviceaccount.com`)

- **Unique ID** (numérico)
- **Client ID** (otro numérico, usado para Domain-Wide Delegation)

Autenticación — 2 opciones

1. **Key JSON descargable** (lo que usamos en IM3):
   - Descargas un archivo JSON con la private key
   - Lo pones en tu env var `GOOGLE_PRIVATE_KEY`
   - El código lo usa para firmar tokens
   - **Riesgo**: si se filtra el JSON, el atacante tiene tu robot. Hay que rotar keys cada 3-6 meses.
2. **Workload Identity** (más seguro, pero más complejo):
   - Solo funciona si corres en Google Cloud (GKE, Cloud Run, etc.)
   - No hay key descargable, se autentica por el entorno
   - No aplica para Railway.

Cómo crear uno (CLI)

```
# 1. Crear service account
gcloud iam service-accounts create MI-SA-NOMBRE \
  --display-name="Mi Service Account" \
  --project=MI-PROYECTO

# 2. Descargar la key JSON
gcloud iam service-accounts keys create ~/mi-sa-key.json \
  --iam-account=MI-SA-NOMBRE@MI-PROYECTO.iam.gserviceaccount.com

# 3. Darle roles que necesita (en el proyecto o org)
gcloud projects add-iam-policy-binding MI-PROYECTO \
  --member="serviceAccount:MI-SA-NOMBRE@MI-PROYECTO.iam.gserviceaccount.com" \
  --role="roles/editor"
```

Cómo los usamos en IM3

En [server/google-calendar.ts]() y similares:

```
const auth = new google.auth.JWT({
  email: process.env.GOOGLE_SERVICE_ACCOUNT_EMAIL,
  key: process.env.GOOGLE_PRIVATE_KEY.replace(/\\n/g, "\n"),
  scopes: ["https://www.googleapis.com/auth/calendar"],
  subject: process.env.GOOGLE_DRIVE_IMPERSONATE, // info@im3systems.com
});
```

El `subject` es donde sucede la magia de Domain-Wide Delegation — siguiente sección.

---

5. Domain-Wide Delegation — La Magia

Un service account **por defecto** solo puede ver los recursos que el proyecto le dejó ver. No puede leer emails de humanos ni crear eventos en calendarios personales.

Domain-Wide Delegation (DWD) cambia eso: autoriza al service account a "hacerse pasar por" cualquier usuario del workspace `im3systems.com`, sin conocer su contraseña.

El flujo mental

```
Tu código:
  "Quiero leer el Gmail de info@im3systems.com"
   |
   ▼
Service Account (im3-diagnostics@...):
  "Soy el robot autorizado del proyecto im3-diagnostics.
   Además, el workspace im3systems.com me autorizó a
   impersonar a sus usuarios con scope gmail.readonly"
   |
   ▼ (firma un JWT con subject=info@im3systems.com)
   ▼
Google:
  "Válido. Acá están los emails de info@im3systems.com"
```

Cómo se configura DWD — 2 lados

Lado 1: Google Cloud (el service account) Ya existe de default — cuando creas un service account, tiene un "Unique Client ID" visible en su detalle. Copia ese número.

Lado 2: Google Workspace Admin Console

1. Ir a admin.google.com con super admin
2. Security → Access and data control → **API controls** → **Manage Domain Wide Delegation**
3. **Add new** → pegar:
   - **Client ID**: el unique ID del service account (ej: `106238472...`)
   - **OAuth Scopes** (lista separada por coma, sin espacios):

     ```
     https://www.googleapis.com/auth/calendar,
     https://www.googleapis.com/auth/drive,
     https://www.googleapis.com/auth/gmail.readonly,
     https://www.googleapis.com/auth/spreadsheets
     ```

4. Save

Desde ese momento, el service account puede impersonar a **cualquier usuario del dominio** — pero solo para los scopes que autorizaste.

Regla de seguridad crítica

NUNCA autorices scopes con `*` o `.full`. Solo los scopes mínimos que necesitas. Si solo vas a leer Gmail, usa `gmail.readonly` (no `gmail.modify`). Si solo vas a leer Drive, usa `drive.readonly`.

Ejemplo práctico

- Para **enviar emails como info@**: scope `gmail.send` + DWD
- Para **leer inbox de info@**: scope `gmail.readonly` + DWD (lo que hacemos en `server/google-gmail.ts`)

- Para **crear eventos en calendar de info@**: scope `calendar` + DWD (lo que hacemos en `server/google-calendar.ts`)
- Para **subir archivos a Drive de info@**: scope `drive` + DWD (lo que hacemos en `server/google-drive.ts`)

Cuándo NO usar DWD

- Si solo necesitas una API con data pública o del service account mismo → no necesitas DWD
- Si el usuario quiere que el sistema acceda a SU gmail personal (no empresarial) → no puedes hacer DWD, usas **OAuth 2.0** (flow de consentimiento)

---

6. Las APIs de Google Workspace

Catálogo práctico de las más usadas. Cada una debe ser **habilitada** en el proyecto Cloud antes de usarla (recuerda el bug del Calendar API).

6.1 Cómo habilitar una API

```
# CLI (más rápido)
gcloud services enable calendar-json.googleapis.com --project=MI-PROYECTO

# UI
https://console.cloud.google.com/apis/library?project=MI-PROYECTO
```

6.2 APIs que usamos (o puedes usar) en IM3

API	Para qué sirve	Scope típico	Se usa en IM3
Calendar API (`calendar-json`)	Crear/leer/eliminar eventos, generar Meet links	`calendar`	✅ `server/google-calendar.ts`
Gmail API (`gmail`)	Leer inbox, enviar emails, buscar por etiquetas	`gmail.readonly`, `gmail.send`	✅ `server/google-gmail.ts` (lectura)
Drive API (`drive`)	Subir/leer archivos, crear carpetas, permisos	`drive`, `drive.readonly`	✅ `server/google-drive.ts`
Sheets API (`sheets`)	Leer/escribir hojas de cálculo	`spreadsheets`	✅ (dentro de google-drive para leer docs)
Docs API (`docs`)	Leer/escribir Google Docs	`documents`	⚠️ opcional
People API (`people`)	Contactos, perfiles	`contacts.readonly`	❌ no usado
Admin SDK (`admin`)	Gestionar usuarios del workspace	`admin.directory.user`	❌ no usado
Meet API	Crear salas de Meet programáticamente	`meetings.space.created`	⚠️ lo hacemos via Calendar

6.3 Scopes — cómo pensarlos

Los scopes son URL. La estructura es:

```
https://www.googleapis.com/auth/<servicio>[.<modificador>]
```

Modificadores comunes:

- `.readonly` → solo lectura
- `.send` → solo envío (no leer)
- `.metadata` → solo metadatos (no contenido)
- sin modificador → acceso total al servicio

Menos es más. Usa siempre el scope más restrictivo que haga la tarea.

6.4 Precios y cuotas

- **Casi todas tienen free tier generoso** para uso típico de un CRM de 100-1000 clientes
- Excepciones que SÍ cuestan: BigQuery queries (por GB procesado), Vertex AI (por token), Cloud Storage (por GB almacenado)
- **Rate limits**: Gmail API tiene ~250 "quota units" por usuario por segundo. Una lectura normal cuesta ~1-5 unidades. En la práctica: para IM3 no vamos a tocar el límite.

6.5 Otras APIs Google (fuera de Workspace)

- **Vertex AI** (Gemini, Imagen) — para IA generativa de Google
- **Cloud Translation** — traducción
- **Cloud Speech-to-Text / Text-to-Speech** — audio (alternativa a Whisper)
- **Cloud Vision** — análisis de imágenes
- **BigQuery** — data warehouse (GB-scale analytics)
- **Cloud SQL** — Postgres/MySQL gestionado (alternativa a Railway)

---

7. gcloud CLI — Tu Herramienta Principal

El Cloud Console (UI web) es útil para explorar, pero para **hacer cosas rápido**, el CLI gana siempre.

7.1 Instalar en Mac

```
brew install --cask google-cloud-sdk
```

Después de instalar, añade al `~/.zshrc` (si no lo hizo automáticamente):

```
source "/opt/homebrew/share/google-cloud-sdk/path.zsh.inc"
source "/opt/homebrew/share/google-cloud-sdk/completion.zsh.inc"
```

7.2 Primer login

```
gcloud auth login info@im3systems.com
```

Te abre el browser, te pide confirmar, y queda autenticado.

7.3 Configurar el "proyecto activo"

La mayoría de comandos asumen un proyecto por default.

```
# Listar proyectos visibles
gcloud projects list

# Activar uno
gcloud config set project im3-diagnostics

# Ver config actual
gcloud config list
```

7.4 Comandos más útiles — chuleta

Proyectos

```
gcloud projects list                                    # listar
gcloud projects describe PROJECT_ID                     # ver detalles
gcloud projects create NEW_ID --organization=ORG_ID     # crear
gcloud beta projects move PROJECT_ID --organization=ID  # mover
```

Organizaciones

```
gcloud organizations list                               # listar orgs
gcloud organizations get-iam-policy ORG_ID              # ver IAM de la org
```

IAM (permisos en un proyecto)

```
# Ver quién tiene qué rol
gcloud projects get-iam-policy PROJECT_ID

# Añadir un principal
gcloud projects add-iam-policy-binding PROJECT_ID \
  --member="user:alguien@dominio.com" \
  --role="roles/editor"

# Remover un principal
gcloud projects remove-iam-policy-binding PROJECT_ID \
  --member="user:alguien@dominio.com" \
  --role="roles/owner"
```

APIs

```
# Listar APIs habilitadas
gcloud services list --enabled --project=PROJECT_ID

# Habilitar
gcloud services enable calendar-json.googleapis.com --project=PROJECT_ID
```

```
# Deshabilitar
gcloud services disable API_NAME --project=PROJECT_ID
```

Service Accounts

```
# Listar
gcloud iam service-accounts list --project=PROJECT_ID

# Crear
gcloud iam service-accounts create sa-name \
  --display-name="Mi SA" \
  --project=PROJECT_ID

# Descargar key
gcloud iam service-accounts keys create ~/key.json \
  --iam-account=sa-name@PROJECT_ID.iam.gserviceaccount.com

# Listar keys
gcloud iam service-accounts keys list \
  --iam-account=sa-name@PROJECT_ID.iam.gserviceaccount.com

# Borrar key vieja
gcloud iam service-accounts keys delete KEY_ID \
  --iam-account=sa-name@PROJECT_ID.iam.gserviceaccount.com
```

Components

```
gcloud components install beta   # para beta commands (como projects move)
gcloud components update         # actualizar todo
```

7.5 Cuándo CLI > UI

- Cuando vas a hacer una acción repetitiva (ej: setup de cliente nuevo)
- Cuando el botón UI no aparece (bug visual, permisos confusos)
- Cuando quieres scriptearlo
- Cuando necesitas hacer la misma operación en 10 proyectos
- Cuando necesitas diagnosticar (`gcloud projects describe` en 1 segundo > abrir UI y buscar)

7.6 Cuándo UI > CLI

- Cuando estás explorando por primera vez
- Cuando necesitas ver gráficos (billing, uso)
- Cuando alguien del equipo no-tech necesita revisar algo

---

8. Anatomía de Este Proyecto (IM3)

Ahora aterrizamos todo lo anterior al **caso real** de IM3. Esto es tu mapa cuando dudes de algo.

8.1 Cuentas y dominios

```
Google Workspace: im3systems.com
  - info@im3systems.com (principal, super admin)
  - mateoobandoangel@gmail.com (personal, YA NO tiene acceso a infra IM3)

Google Cloud:
  - Organization: im3systems.com (ID 980700249203)
  - Project: im3-diagnostics (ID 634399080625)
    Parent: organization 980700249203  ← desde 2026-04-14 (antes era huérfano)
    Owner: info@im3systems.com (único)

Service Account:
  - im3-diagnostics@im3-diagnostics.iam.gserviceaccount.com
  - Key JSON: en env var GOOGLE_PRIVATE_KEY (Railway)
```

8.2 APIs habilitadas en `im3-diagnostics`

- Calendar API (habilitada 2026-04-14 tras diagnóstico)
- Gmail API
- Drive API
- Sheets API

8.3 Domain-Wide Delegation

Configurada en `admin.google.com` → API controls:

- Client ID del service account autorizado para impersonar usuarios de `im3systems.com`
- Scopes autorizados: `calendar`, `drive`, `gmail.readonly`, `spreadsheets`
- Usuario que se impersona en el código: `info@im3systems.com`

8.4 Dónde viven las credenciales

En Railway (var de entorno del proyecto IM3 Website):

- `GOOGLE_SERVICE_ACCOUNT_EMAIL` → email del SA
- `GOOGLE_PRIVATE_KEY` → la key JSON (como string, con `\n` literales)
- `GOOGLE_DRIVE_IMPERSONATE` → `info@im3systems.com` (el usuario a impersonar)
- `GOOGLE_DRIVE_FOLDER_ID` → ID de la carpeta raíz donde se crean subcarpetas de clientes

8.5 Código que las usa

- server/google-calendar.ts — crear eventos + Meet links
- server/google-drive.ts — carpetas cliente, upload/read de docs
- server/google-gmail.ts — sync de Gmail inbox al CRM

8.6 Flujo completo cuando alguien llena el diagnóstico

```
1. Cliente abre /booking → llena form → POST /api/diagnostic
2. routes.ts guarda diagnostic en DB
3. google-drive.ts crea carpeta cliente en Drive (como info@im3systems.com)
4. google-calendar.ts crea evento + Meet link (como info@im3systems.com)
   → info@ recibe la invitación en SU calendar
   → cliente recibe la invitación en el email que puso
5. routes.ts programa 7 emails con Claude (sent_emails, status=pending)
```

```
6. El primer email "confirmacion" se envía inmediato via Resend (no Google)
7. Cada 15 min el cron email-queue procesa los pending
```

Todos los pasos 3-4 dependen de la arquitectura Cloud + DWD. Si rompes DWD o deshabilitas una API, se cae esa parte.

---

9. Playbook de Replicación a Cliente Nuevo

Tu escenario real: **"Cliente ACME nos pide un CRM automatizado como el de IM3"**.

Opción A (recomendada): TODO en la cuenta Cloud de IM3

El cliente NO necesita tener Workspace ni cuenta Google. Nosotros usamos **nuestra** infraestructura y le mandamos emails desde IM3.

- Ventaja: simple, barato, cliente no depende de setup suyo
- Desventaja: los emails salen de nuestro dominio, no del suyo

Flujo: crear proyecto "acme-crm" dentro de la org de IM3, usar los mismos service accounts y credentials.

Opción B (avanzada): separación total — cliente dueño de su infra

El cliente tiene Workspace propio (`@acme.com`), nosotros le configuramos la integración con su Google.

- Ventaja: datos del cliente no pasan por nosotros, emails salen de `@acme.com` , cliente puede "llevarse" la implementación
- Desventaja: más trabajo inicial, depende de que el cliente mantenga billing

Checklist Opción B (el caso más complejo)

Paso 0 — Confirma requisitos con el cliente:

- [ ] ¿Tienen Workspace activo? (Si solo usan `@gmail.com` , HAY que convencerlos de migrar a Workspace — no funciona con gmail personal)
- [ ] ¿Quién es el super admin? Necesitamos acceso temporal o que ellos ejecuten 2-3 pasos.
- [ ] ¿Qué quieren integrar? (Gmail sync? Calendar? Drive? los 3?)
- [ ] ¿Qué dominio tendrá el CRM? (subdominio de acme.com? hub.im3systems.com/acme?)

Paso 1 — Crear proyecto Cloud en su org:

```
# Logueado con un usuario admin de la org del cliente
gcloud auth login admin@acme.com
gcloud projects create acme-crm \
  --organization=ORG_ID_DE_ACME \
  --name="ACME CRM"
gcloud config set project acme-crm
```

Paso 2 — Habilitar APIs necesarias:

```
gcloud services enable \
  calendar-json.googleapis.com \
  gmail.googleapis.com \
  drive.googleapis.com \
```

```
  sheets.googleapis.com \
  --project=acme-crm
```

Paso 3 — Crear service account:

```
gcloud iam service-accounts create acme-crm-sa \
  --display-name="ACME CRM Service Account" \
  --project=acme-crm

# Descargar key
gcloud iam service-accounts keys create ~/acme-crm-key.json \
  --iam-account=acme-crm-sa@acme-crm.iam.gserviceaccount.com
```

Paso 4 — Configurar Domain-Wide Delegation (el cliente debe hacer esto, o darte acceso temporal a admin.google.com):

1. admin.google.com con super admin de `@acme.com`
2. Security → API controls → Domain-wide delegation → Add
3. Client ID: obtenerlo con:

   ```
   gcloud iam service-accounts describe acme-crm-sa@acme-
   crm.iam.gserviceaccount.com \
     --format="value(oauth2ClientId)"
   ```

4. Scopes (mínimos):

   ```
   https://www.googleapis.com/auth/calendar,
   https://www.googleapis.com/auth/drive,
   https://www.googleapis.com/auth/gmail.readonly,
   https://www.googleapis.com/auth/spreadsheets
   ```

5. Save

Paso 5 — Configurar env vars del cliente: En tu backend (Railway/donde sea), para ese cliente:

```
GOOGLE_SERVICE_ACCOUNT_EMAIL=acme-crm-sa@acme-crm.iam.gserviceaccount.com
GOOGLE_PRIVATE_KEY=<contenido de acme-crm-key.json, la propiedad private_key>
GOOGLE_DRIVE_IMPERSONATE=admin@acme.com
GOOGLE_DRIVE_FOLDER_ID=<ID de una carpeta que creaste en Drive para organizarlo>
```

Paso 6 — Validar con el script (adaptado del de IM3):

```
npx tsx scripts/diagnose-calendar.ts
```

Debe devolver ✅ `CALENDAR FUNCIONA`.

Paso 7 — Limpiar key local:

```
rm ~/acme-crm-key.json   # borrar del disco, ya vive en Railway
```

Paso 8 — Documentar para el cliente qué tienen y cómo recuperar acceso si algo falla.

---

10. Seguridad y Recovery

10.1 Principio: Least Privilege

Siempre dale al service account **el scope más restrictivo** que haga la tarea. Si solo lees Gmail, usa `gmail.readonly`. Si solo subes archivos a Drive, usa `drive.file` (solo archivos que el SA crea).

10.2 Rotación de keys

Las keys JSON deberían rotarse cada **3-6 meses** como hábito, o inmediatamente si sospechas compromiso.

Proceso:

```
# 1. Crear nueva key
gcloud iam service-accounts keys create ~/new-key.json \
  --iam-account=sa@project.iam.gserviceaccount.com

# 2. Actualizar env var en Railway (hot reload)
# copia el contenido de private_key a GOOGLE_PRIVATE_KEY

# 3. Verificar que el sistema sigue funcionando
npx tsx scripts/diagnose-calendar.ts

# 4. Listar keys activas
gcloud iam service-accounts keys list \
  --iam-account=sa@project.iam.gserviceaccount.com

# 5. Borrar la vieja (ID viene del paso 4)
gcloud iam service-accounts keys delete OLD_KEY_ID \
  --iam-account=sa@project.iam.gserviceaccount.com
```

10.3 2FA y Recovery en cuentas humanas

Para cada cuenta admin (ej: `info@im3systems.com`), configurar:

- 2FA obligatorio (authenticator app + backup codes)
- Recovery email (puede ser tu personal)
- Recovery phone
- Password en password manager

Ver link directos en la Sección "Checklist de security setup" de la sesión anterior.

10.4 Super admins

Cada workspace necesita **al menos 2 super admins** — nunca 1 solo. Si te pasa algo, el negocio debe poder seguir.

Recomendación:

- Super admin 1: `info@im3systems.com` (tú, el operador principal)
- Super admin 2: `mateo@im3systems.com` (una cuenta adicional tuya con 2FA distinto, o la cuenta de un socio de confianza)

10.5 Domain Restricted Sharing

Mantenla **activa** por default. Solo relájala si tienes una razón muy concreta (consultor temporal con scope acotado).

10.6 Auditoría

Google Cloud tiene logs de todo (quien hizo qué y cuándo):

- Cloud Audit Logs — busca "admin activity"
- `gcloud logging read "severity>=WARNING" --limit=50 --project=PROJECT_ID`

Si algo raro pasa, ahí está el registro.

10.7 Billing

- Configura **alertas de billing** para cada proyecto (ej: aviso si se supera $50/mes)
- Enlace: https://console.cloud.google.com/billing → Budgets & alerts
- Previene sorpresas si algún cron se vuelve loco consumiendo API

---

11. Troubleshooting Frecuente

11.1 Error: `403 Google X API has not been used in project Y before or it is disabled`

Causa: La API no está habilitada en el proyecto. **Fix**:

```
gcloud services enable X.googleapis.com --project=PROJECT_ID
```

Puede tomar 2-5 min en propagar.

11.2 Error: `IAM policy update failed: iam.allowedPolicyMemberDomains`

Causa: Domain Restricted Sharing impide añadir principals fuera del dominio. **Fix** (si es intencional): relajar la policy temporalmente (ver Sección 3). **Fix** (si no): usa un email del dominio correcto.

11.3 Error: `invalid_grant: Invalid JWT Signature` o `Invalid grant: account not found`

Causa: El service account está intentando impersonar un usuario pero no está autorizado. **Checklist**:

- ¿El usuario impersonado existe en el Workspace? (si dice `test@im3systems.com` y ese usuario no existe, falla)
- ¿El Client ID del SA está registrado en Domain-Wide Delegation del workspace?
- ¿El scope pedido en el código está incluido en los scopes autorizados?

11.4 Error al migrar proyecto: `You do not have permission to perform this action`

Causa: falta rol `Project Mover` o `Project Creator`. **Fix**:

```
# Dar Project Mover a nivel organización al usuario que está migrando
gcloud organizations add-iam-policy-binding ORG_ID \
```

```
  --member="user:info@im3systems.com" \
  --role="roles/resourcemanager.projectMover"
```

11.5 Service account key expira o filtrada

Fix: rotar (ver Sección 10.2).

11.6 Error: `Quota exceeded` en una API

Causa: superaste el rate limit. **Fix inmediato**: reintentar con backoff (el código ya lo hace para la mayoría). **Fix largo plazo**: solicitar aumento de quota en Cloud Console → IAM & Admin → Quotas.

11.7 Google Meet no genera link en el evento creado

Causa 1: no incluiste `conferenceData` en el request. **Causa 2**: el user impersonado no tiene licencia de Meet activa (algunas licencias de Workspace Basic no incluyen Meet avanzado). **Causa 3**: la conferencia quedó en `status: pending` y tu código no hace polling. Ver server/google-calendar.ts:138-172 — ahí hay retry con backoff.

11.8 No veo el proyecto en la UI del Cloud Console

Causa 1: el selector superior está apuntando a otra organización. **Causa 2**: tu cuenta no tiene permisos en ese proyecto. **Fix**: verificar ambos con `gcloud projects list` (muestra los que TÚ ves).

---

12. Glosario

Término	Explicación corta
Principal	Una identidad que puede tener permisos (usuario, grupo, service account, dominio)
Role	Conjunto de permisos predefinidos (ej: Owner, Editor, Viewer)
Role binding	La asignación de un role a un principal en un recurso específico
Policy	La suma de todos los role bindings de un recurso
IAM	Identity and Access Management — el sistema de permisos de Google Cloud
ADC	Application Default Credentials — credenciales que `gcloud` usa automáticamente si no se especifican
Service Account (SA)	Identidad robot, no humana, usada por código
Workload Identity	Alternativa moderna a keys JSON, solo funciona dentro de Google Cloud
Domain-Wide Delegation (DWD)	Autorización en Workspace para que un SA impersone usuarios del dominio
Impersonation / Subject claim	Cuando un SA actúa "como si fuera" otro usuario, via DWD
OAuth scope	Permiso granular sobre una API específica (ej: `gmail.readonly`)

Consent screen	Pantalla de permisos que ve un usuario cuando una app pide acceso a su cuenta (NO aplica con SA+DWD)
Client ID	Identificador público de una aplicación OAuth o un service account
Organization / Org	Capa superior de la jerarquía de Google Cloud, asociada a un Workspace
Folder	Sub-agrupación de proyectos dentro de una org, opcional
Project	Contenedor de APIs, recursos, billing en Cloud
Billing Account	Cuenta que paga los usos. Se vincula a uno o más proyectos
Quota	Límite de uso de una API (por minuto, por día, por proyecto)
Rate limit	Cuántas requests puedes hacer por unidad de tiempo
Workspace	Suite de apps empresariales (Gmail, Calendar, Drive, Meet) de pago
Super admin	Admin máximo de un workspace — gestiona todos los usuarios
Domain Restricted Sharing	Policy que limita qué dominios pueden recibir permisos en tu org
API Library	Catálogo de APIs que puedes habilitar en un proyecto
Cloud Audit Logs	Registro de todas las acciones en tu infraestructura
gcloud	CLI oficial de Google Cloud
gsutil	CLI específico para Cloud Storage (viene con gcloud)
bq	CLI para BigQuery (viene con gcloud)
gcloud components	Módulos adicionales del CLI (beta, alpha, kubectl, etc.)
JWT	JSON Web Token — formato de token firmado usado en autenticación
ID Token vs Access Token	ID token dice *quién* eres; Access token dice *qué puedes hacer*
Service Account User	Role que permite a un humano "usar" un SA (ej: para deploy)
Service Account Token Creator	Role que permite generar tokens en nombre de un SA (impersonation)

Preguntas de autoevaluación

Cuando termines de estudiar, deberías poder responder sin consultar:

1. ¿Por qué el service account de IM3 puede leer emails de `info@im3systems.com`? → Porque tiene Domain-Wide Delegation configurada en el workspace `im3systems.com`, autorizando el Client ID del SA con el scope `gmail.readonly`.
2. ¿Qué pasaría si movemos el proyecto a otra organización? → Las APIs habilitadas, service accounts y sus keys siguen funcionando (viven dentro del proyecto). Pero el proyecto hereda las policies de

la nueva org, y los usuarios que heredaban permisos por la org anterior pierden acceso.

3. ¿Qué diferencia hay entre Owner del proyecto y super admin del workspace? → Owner del proyecto Cloud gestiona ese proyecto específico (IAM, APIs, recursos). Super admin del workspace gestiona usuarios del dominio, Admin Console, Domain-Wide Delegation. Son mundos separados — puedes ser una cosa sin ser la otra.

4. Si un cliente nuevo nos pide integrar su Gmail, ¿qué 5 pasos siguen? → (1) Crear proyecto en la org del cliente, (2) Habilitar Gmail API, (3) Crear Service Account + key, (4) Configurar Domain-Wide Delegation con el Client ID del SA en el admin console del cliente con scope `gmail.readonly`, (5) Configurar env vars en nuestro backend.

5. Si `gcloud beta projects move` falla con "permission denied", ¿qué se revisa primero? → Que el usuario logueado (`gcloud auth list`) tenga el rol `roles/resourcemanager.projectMover` sobre el proyecto Y `roles/resourcemanager.projectCreator` sobre la organización destino.

Recursos oficiales (para profundizar)

- Google Cloud Fundamentals (curso gratis)
- IAM documentation
- Service Account docs
- Domain-Wide Delegation setup
- gcloud CLI reference
- Workspace API docs